

Tasks to Perform

1. Understand the dataset:

```
In [117... !pip install scipy
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
from scipy.stats import kruskal
```

Requirement already satisfied: scipy in c:\users\rutuja\appdata\local\programs\python\python310\lib\site-packages (1.15.3)

Requirement already satisfied: numpy<2.5,>=1.23.5 in c:\users\rutuja\appdata\local\programs\python\python310\lib\site-packages (from scipy) (2.2.6)

1.1 Import the dataset

```
In [116... df = pd.read_csv("D:/Data Analyst Course/Python/Project/311_Service_Requests_fro

C:\Users\Rutuja\AppData\Local\Temp\ipykernel_9884\4249846094.py:1: DtypeWarning:
Columns (48,49) have mixed types. Specify dtype option on import or set low_memory=False.
df = pd.read_csv("D:/Data Analyst Course/Python/Project/311_Service_Requests_from_2010_to_Present.csv")
```

1.2 Visualize the dataset

```
In [3]: df.head()
```

Out[3]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	Loc
0	32310363	12/31/2015 11:59:45 PM	01-01-2016 00:55	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party	Stree
1	32309934	12/31/2015 11:59:44 PM	01-01-2016 01:26	NYPD	New York City Police Department	Blocked Driveway	No Access	Stree
2	32309159	12/31/2015 11:59:29 PM	01-01-2016 04:51	NYPD	New York City Police Department	Blocked Driveway	No Access	Stree
3	32305098	12/31/2015 11:57:46 PM	01-01-2016 07:43	NYPD	New York City Police Department	Illegal Parking	Commercial Overnight Parking	Stree
4	32306529	12/31/2015 11:56:58 PM	01-01-2016 03:24	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk	Stree

5 rows × 53 columns

In [4]: `df.tail()`

Out[4]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	I
364553	29609918	01-01-2015 00:04	01-01-2015 10:22	NYPD	New York City Police Department	Illegal Parking	Blocked Hydrant	S
364554	29608392	01-01-2015 00:04	01-01-2015 02:25	NYPD	New York City Police Department	Noise - Vehicle	Car/Truck Horn	S
364555	29607589	01-01-2015 00:01	01-01-2015 00:20	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party	S
364556	29610889	01-01-2015 00:01	01-01-2015 02:42	NYPD	New York City Police Department	Blocked Driveway	No Access	S
364557	29611816	01-01-2015 00:00	01-01-2015 02:47	NYPD	New York City Police Department	Blocked Driveway	No Access	S

5 rows × 53 columns

In [5]: `df.sample(5)`

Out[5]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descri
4952	32274743	12/26/2015 06:14:35 AM	12/26/2015 08:21:40 AM	NYPD	New York City Police Department	Illegal Parking	Comme Overr Par
139446	31376824	08/23/2015 06:20:32 AM	08/23/2015 07:26:30 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	l Tal
37630	32049337	11/23/2015 08:34:53 AM	11/23/2015 02:50:05 PM	NYPD	New York City Police Department	Blocked Driveway	P Ac
19966	32184169	12-11- 2015 11:31	12-11- 2015 20:02	NYPD	New York City Police Department	Blocked Driveway	No Ac
211557	30896478	06/20/2015 10:29:52 PM	NaN	NYPD	New York City Police Department	Noise - Commercial	l Music/F

5 rows × 53 columns



1.3 Print the columns of the DataFrame

In [6]: `df.columns`

```
Out[6]: Index(['Unique Key', 'Created Date', 'Closed Date', 'Agency', 'Agency Name',
              'Complaint Type', 'Descriptor', 'Location Type', 'Incident Zip',
              'Incident Address', 'Street Name', 'Cross Street 1', 'Cross Street 2',
              'Intersection Street 1', 'Intersection Street 2', 'Address Type',
              'City', 'Landmark', 'Facility Type', 'Status', 'Due Date',
              'Resolution Description', 'Resolution Action Updated Date',
              'Community Board', 'Borough', 'X Coordinate (State Plane)',
              'Y Coordinate (State Plane)', 'Park Facility Name', 'Park Borough',
              'School Name', 'School Number', 'School Region', 'School Code',
              'School Phone Number', 'School Address', 'School City', 'School State',
              'School Zip', 'School Not Found', 'School or Citywide Complaint',
              'Vehicle Type', 'Taxi Company Borough', 'Taxi Pick Up Location',
              'Bridge Highway Name', 'Bridge Highway Direction', 'Road Ramp',
              'Bridge Highway Segment', 'Garage Lot Name', 'Ferry Direction',
              'Ferry Terminal Name', 'Latitude', 'Longitude', 'Location'],
              dtype='object')
```

```
In [7]: for col in df.columns:
        print(col)
```

Unique Key
Created Date
Closed Date
Agency
Agency Name
Complaint Type
Descriptor
Location Type
Incident Zip
Incident Address
Street Name
Cross Street 1
Cross Street 2
Intersection Street 1
Intersection Street 2
Address Type
City
Landmark
Facility Type
Status
Due Date
Resolution Description
Resolution Action Updated Date
Community Board
Borough
X Coordinate (State Plane)
Y Coordinate (State Plane)
Park Facility Name
Park Borough
School Name
School Number
School Region
School Code
School Phone Number
School Address
School City
School State
School Zip
School Not Found
School or Citywide Complaint
Vehicle Type
Taxi Company Borough
Taxi Pick Up Location
Bridge Highway Name
Bridge Highway Direction
Road Ramp
Bridge Highway Segment
Garage Lot Name
Ferry Direction
Ferry Terminal Name
Latitude
Longitude
Location

1.4 Identify the shape of the dataset

```
In [8]: df.shape
```

Out[8]: (364558, 53)

1.5 Identify the variables with null values

In [9]: `df.isnull().sum()`

```

Out[9]: Unique Key          0
        Created Date       0
        Closed Date       2381
        Agency            0
        Agency Name       0
        Complaint Type    0
        Descriptor       6501
        Location Type     133
        Incident Zip      2998
        Incident Address  51699
        Street Name       51699
        Cross Street 1    57188
        Cross Street 2    57805
        Intersection Street 1 313438
        Intersection Street 2 314046
        Address Type      3252
        City              2997
        Landmark          364183
        Facility Type     2389
        Status            0
        Due Date          3
        Resolution Description 0
        Resolution Action Updated Date 2402
        Community Board   0
        Borough           0
        X Coordinate (State Plane) 4030
        Y Coordinate (State Plane) 4030
        Park Facility Name 0
        Park Borough      0
        School Name       0
        School Number     0
        School Region     1
        School Code       1
        School Phone Number 0
        School Address    0
        School City       0
        School State      0
        School Zip        1
        School Not Found  0
        School or Citywide Complaint 364558
        Vehicle Type      364558
        Taxi Company Borough 364558
        Taxi Pick Up Location 364558
        Bridge Highway Name 364261
        Bridge Highway Direction 364261
        Road Ramp         364296
        Bridge Highway Segment 364296
        Garage Lot Name   364558
        Ferry Direction   364557
        Ferry Terminal Name 364556
        Latitude          4030
        Longitude         4030
        Location          4030
        dtype: int64

```

```

In [10]: null_values = df.isnull().sum()
         null_values.sort_values(ascending=False)

```

```

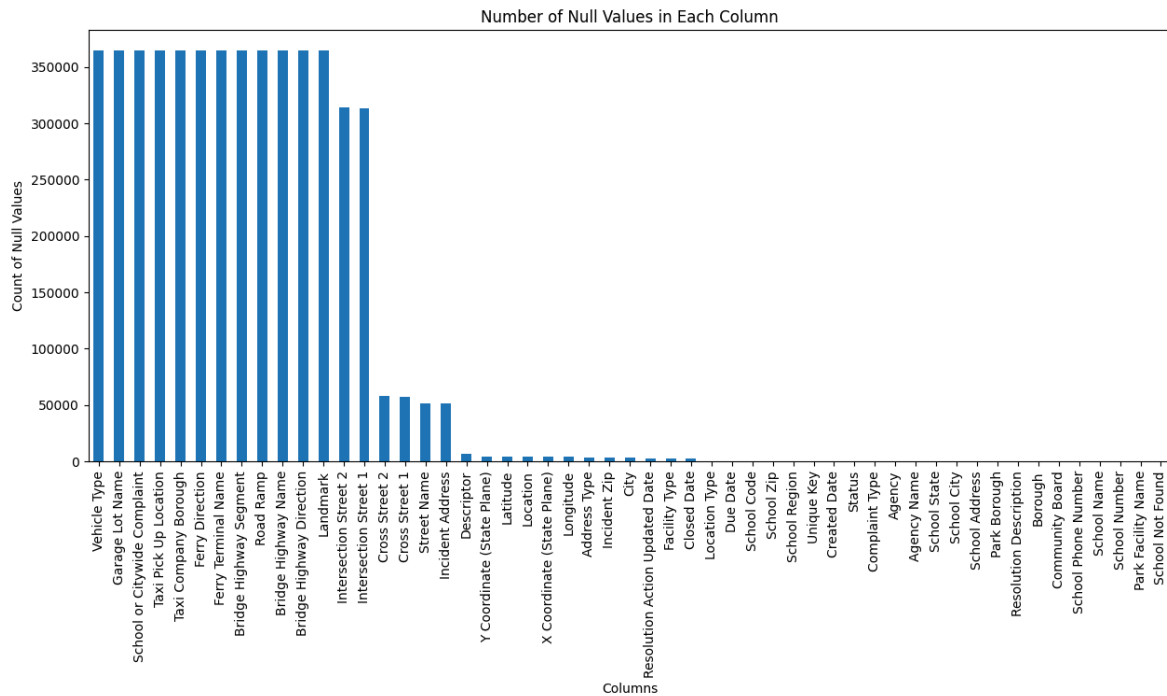
Out[10]: Vehicle Type          364558
         Garage Lot Name       364558
         School or Citywide Complaint 364558
         Taxi Pick Up Location  364558
         Taxi Company Borough  364558
         Ferry Direction        364557
         Ferry Terminal Name     364556
         Bridge Highway Segment 364296
         Road Ramp               364296
         Bridge Highway Name     364261
         Bridge Highway Direction 364261
         Landmark                364183
         Intersection Street 2   314046
         Intersection Street 1   313438
         Cross Street 2          57805
         Cross Street 1          57188
         Street Name             51699
         Incident Address        51699
         Descriptor              6501
         Y Coordinate (State Plane) 4030
         Latitude                4030
         Location                4030
         X Coordinate (State Plane) 4030
         Longitude               4030
         Address Type            3252
         Incident Zip            2998
         City                    2997
         Resolution Action Updated Date 2402
         Facility Type           2389
         Closed Date             2381
         Location Type           133
         Due Date                3
         School Code             1
         School Zip              1
         School Region           1
         Unique Key              0
         Created Date            0
         Status                  0
         Complaint Type          0
         Agency                  0
         Agency Name             0
         School State            0
         School City             0
         School Address          0
         Park Borough            0
         Resolution Description   0
         Borough                 0
         Community Board         0
         School Phone Number     0
         School Name             0
         School Number           0
         Park Facility Name      0
         School Not Found        0
dtype: int64

```

2. Perform basic data exploratory analysis:

2.1 Draw a frequency plot to show the number of null values in each column of the DataFrame

```
In [11]: plt.figure(figsize=(15,6))
null_values.sort_values(ascending=False).plot(kind='bar')
plt.title("Number of Null Values in Each Column")
plt.xlabel("Columns")
plt.ylabel("Count of Null Values")
plt.xticks(rotation=90)
plt.show()
```



2.2 Missing value treatment

```
In [12]: df.isnull().sum()
```

```

Out[12]: Unique Key          0
Created Date                0
Closed Date                2381
Agency                    0
Agency Name                0
Complaint Type              0
Descriptor                 6501
Location Type              133
Incident Zip               2998
Incident Address           51699
Street Name                51699
Cross Street 1             57188
Cross Street 2             57805
Intersection Street 1      313438
Intersection Street 2      314046
Address Type               3252
City                       2997
Landmark                   364183
Facility Type              2389
Status                     0
Due Date                   3
Resolution Description      0
Resolution Action Updated Date 2402
Community Board            0
Borough                    0
X Coordinate (State Plane) 4030
Y Coordinate (State Plane) 4030
Park Facility Name         0
Park Borough               0
School Name                0
School Number              0
School Region              1
School Code                1
School Phone Number        0
School Address              0
School City                0
School State                0
School Zip                 1
School Not Found           0
School or Citywide Complaint 364558
Vehicle Type               364558
Taxi Company Borough       364558
Taxi Pick Up Location      364558
Bridge Highway Name        364261
Bridge Highway Direction    364261
Road Ramp                  364296
Bridge Highway Segment     364296
Garage Lot Name            364558
Ferry Direction            364557
Ferry Terminal Name        364556
Latitude                   4030
Longitude                  4030
Location                   4030
dtype: int64

```

2.2.1 Remove the records whose Closed Date values are null

```
In [13]: df = df[df["Closed Date"].notnull()]
```

```
df["Closed Date"].isnull().sum()
```

```
Out[13]: np.int64(0)
```

2.3 Analyze the date column, and remove entries that have an incorrect timeline

```
In [ ]: df["Created Date"] = pd.to_datetime(df["Created Date"])
df["Closed Date"] = pd.to_datetime(df["Closed Date"])
```

```
In [15]: df[["Created Date", "Closed Date"]].dtypes
```

```
Out[15]: Created Date    datetime64[ns]
Closed Date           object
dtype: object
```

```
In [23]: df = df[df["Closed Date"] >= df["Created Date"]]
df.head(2)
```

```
Out[23]:
```

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	Locat
0	32310363	2015-12-31 23:59:45	2016-01-01 00:55:00	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party	Street/
1	32309934	2015-12-31 23:59:44	2016-01-01 01:26:00	NYPD	New York City Police Department	Blocked Driveway	No Access	Street/

2 rows × 53 columns



2.3.1 Calculate the time elapsed in closed and creation date

```
In [21]: df["Closed Date"] = pd.to_datetime(
    df["Closed Date"],
    errors="coerce",
    format="mixed"
)
```

```
In [22]: print(df[["Created Date", "Closed Date"]].dtypes)
```

```
Created Date    datetime64[ns]
Closed Date    datetime64[ns]
dtype: object
```

```
In [24]: df["Request Closing Time"] = df["Closed Date"] - df["Created Date"]
df[["Created Date", "Closed Date", "Request Closing Time"]].head()
```

Out[24]:

	Created Date	Closed Date	Request Closing Time
0	2015-12-31 23:59:45	2016-01-01 00:55:00	0 days 00:55:15
1	2015-12-31 23:59:44	2016-01-01 01:26:00	0 days 01:26:16
2	2015-12-31 23:59:29	2016-01-01 04:51:00	0 days 04:51:31
3	2015-12-31 23:57:46	2016-01-01 07:43:00	0 days 07:45:14
4	2015-12-31 23:56:58	2016-01-01 03:24:00	0 days 03:27:02

In [20]: `print(df[["Created Date", "Closed Date"]].dtypes)`

```
Created Date    datetime64[ns]
Closed Date      object
dtype: object
```

2.3.2 Convert the calculated date to seconds to get a better representation

In [25]: `df["Request Closing Time Seconds"] = df["Request Closing Time"].dt.total_seconds()
df["Request Closing Time Seconds"].head()`

Out[25]:

```
0      3315.0
1      5176.0
2     17491.0
3     27914.0
4     12422.0
Name: Request Closing Time Seconds, dtype: float64
```

2.3.3 View the descriptive statistics for the newly created column

In [26]: `df["Request Closing Time Seconds"].describe()`

Out[26]:

```
count    3.621770e+05
mean     1.511330e+04
std       2.110251e+04
min       6.000000e+01
25%      4.536000e+03
50%      9.609000e+03
75%     1.887900e+04
max     2.134342e+06
Name: Request Closing Time Seconds, dtype: float64
```

2.3.4 Check the number of null values in the Complaint_Type and City columns

In [27]: `df[["Complaint Type", "City"]].isnull().sum()`

Out[27]:

```
Complaint Type    0
City              674
dtype: int64
```

2.3.5 Impute the NA value with Unknown City

```
In [29]: df["City"] = df["City"].fillna("Unknown City")
df["City"].isnull().sum()
```

```
Out[29]: np.int64(0)
```

2.3.6 Draw a frequency plot for the complaints in each city

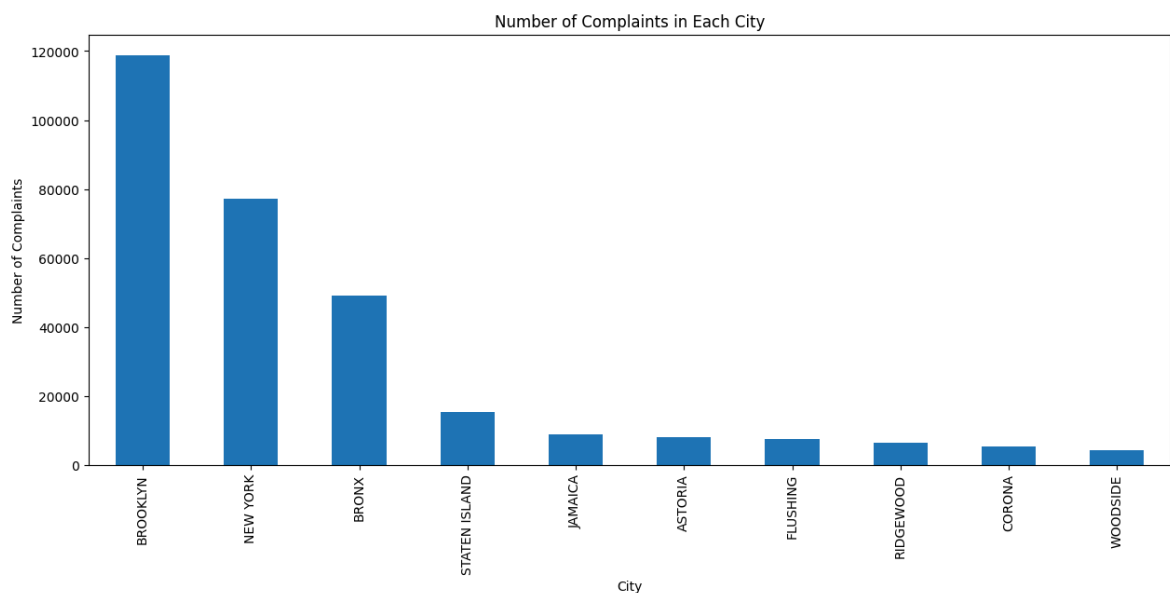
```
In [32]: plt.figure(figsize=(15,6))

df["City"].value_counts().head(10).plot(kind="bar")

plt.title("Number of Complaints in Each City")
plt.xlabel("City")
plt.ylabel("Number of Complaints")

# plt.xticks(rotation=90)

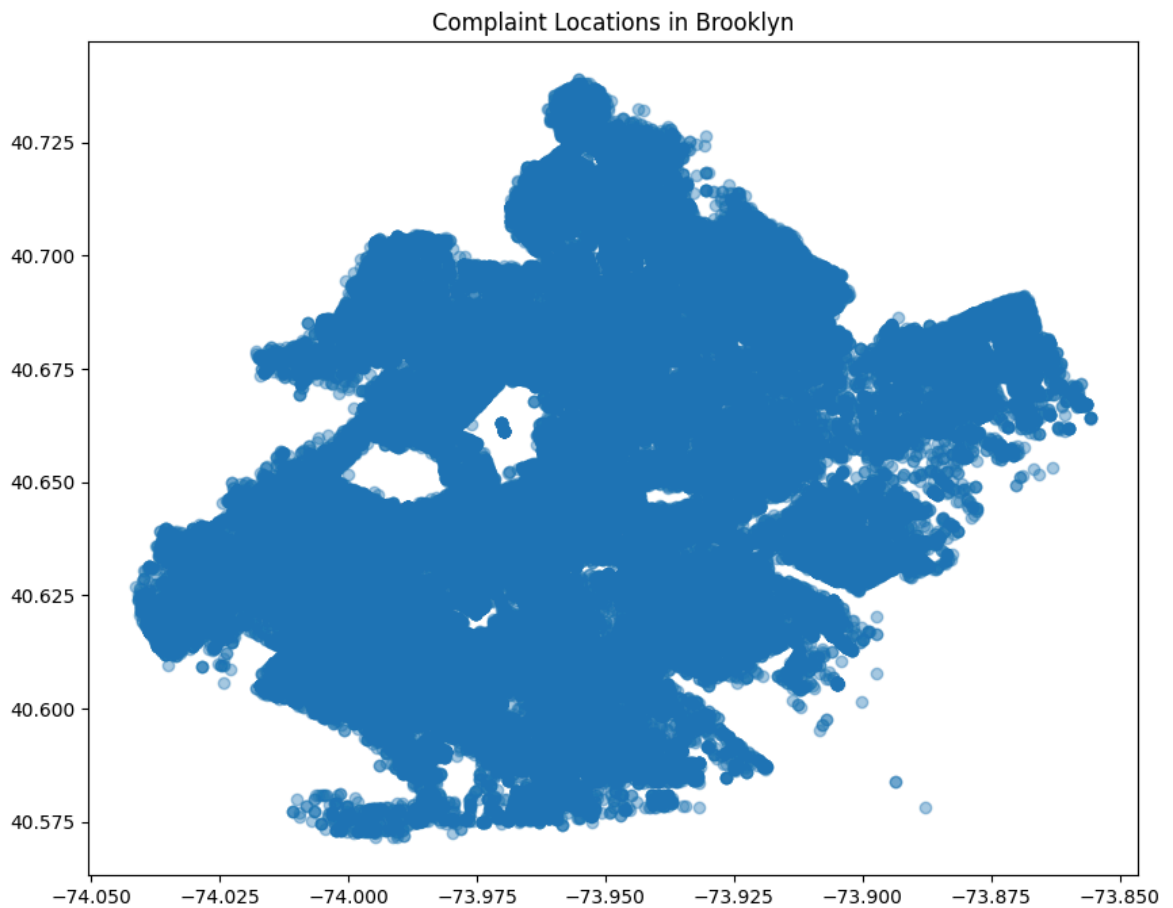
plt.show()
```



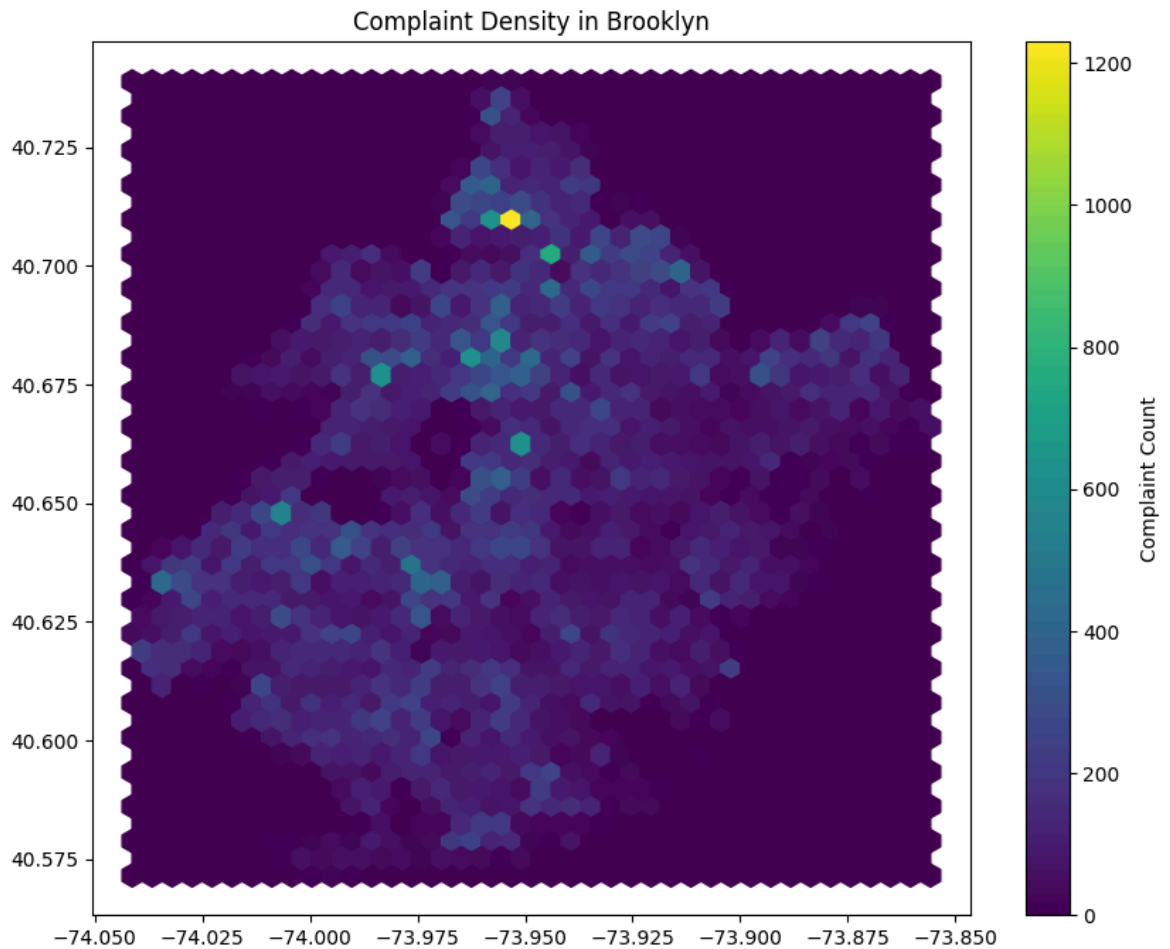
2.3.7 Create a scatter and hexbin plot of the concentration of complaints across Brooklyn

```
In [33]: brooklyn = df[df["Borough"] == "BROOKLYN"]
```

```
In [35]: plt.figure(figsize=(10,8))
plt.scatter(
    brooklyn["Longitude"],
    brooklyn["Latitude"],
    alpha=0.4
)
plt.title("Complaint Locations in Brooklyn")
plt.show()
```



```
In [37]: plt.figure(figsize=(10,8))
plt.hexbin(
    brooklyn["Longitude"],
    brooklyn["Latitude"],
    gridsize=40,
    # cmap="viridis"
)
plt.colorbar(label="Complaint Count")
plt.title("Complaint Density in Brooklyn")
plt.show()
```



3. Find major types of complaints:

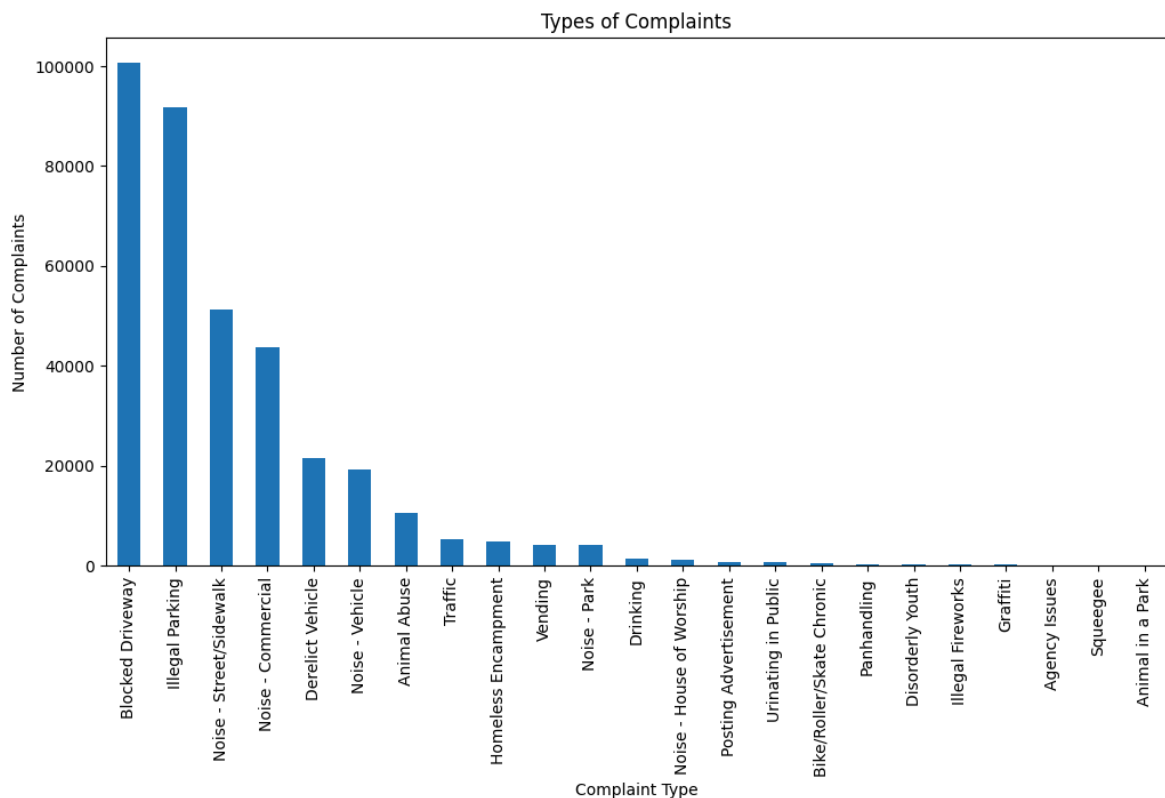
3.1 Plot a bar graph to show the types of complaints

```
In [38]: complaint_count = df["Complaint Type"].value_counts()  
print(complaint_count)
```

Complaint Type	
Blocked Driveway	100624
Illegal Parking	91716
Noise - Street/Sidewalk	51139
Noise - Commercial	43751
Derelict Vehicle	21518
Noise - Vehicle	19301
Animal Abuse	10530
Traffic	5196
Homeless Encampment	4879
Vending	4185
Noise - Park	4089
Drinking	1404
Noise - House of Worship	1068
Posting Advertisement	679
Urinating in Public	641
Bike/Roller/Skate Chronic	475
Panhandling	325
Disorderly Youth	315
Illegal Fireworks	172
Graffiti	157
Agency Issues	8
Squeegee	4
Animal in a Park	1

Name: count, dtype: int64

```
In [39]: plt.figure(figsize=(12,6))
complaint_count.plot(kind='bar')
plt.title("Types of Complaints")
plt.xlabel("Complaint Type")
plt.ylabel("Number of Complaints")
plt.show()
```



3.2 Check the frequency of various types of complaints for New York City

```
In [42]: nyc_complaints = nyc["Complaint Type"].value_counts()
print(nyc_complaints)
```

```
Complaint Type
Noise - Street/Sidewalk    22245
Noise - Commercial         18686
Illegal Parking            14549
Noise - Vehicle            6294
Homeless Encampment        3060
Blocked Driveway           2705
Vending                    2638
Animal Abuse               1941
Traffic                    1769
Noise - Park               1243
Derelict Vehicle           695
Drinking                   321
Urinating in Public        264
Bike/Roller/Skate Chronic  254
Noise - House of Worship   222
Panhandling                206
Disorderly Youth           81
Posting Advertisement       49
Illegal Fireworks           38
Graffiti                   25
Squeegee                    4
Name: count, dtype: int64
```

3.3 Find the top 10 complaint types

```
In [43]: top10 = df["Complaint Type"].value_counts().head(10)
print(top10)
```

```
Complaint Type
Blocked Driveway    100624
Illegal Parking     91716
Noise - Street/Sidewalk  51139
Noise - Commercial  43751
Derelict Vehicle    21518
Noise - Vehicle     19301
Animal Abuse        10530
Traffic             5196
Homeless Encampment  4879
Vending             4185
Name: count, dtype: int64
```

3.4 Display the various types of complaints in each city

```
In [55]: df.groupby("City")["Complaint Type"].value_counts()
```

```
Out[55]: City      Complaint Type
ARVERNE  Illegal Parking      62
          Blocked Driveway     50
          Animal Abuse         46
          Derelict Vehicle     32
          Noise - Street/Sidewalk 29
          ...
Woodside  Illegal Parking     124
          Blocked Driveway     27
          Derelict Vehicle      8
          Noise - Street/Sidewalk 5
          Noise - Commercial    2
Name: count, Length: 792, dtype: int64
```

3.5 Create a DataFrame, df_new, which contains cities as columns and complaint types in rows

```
In [59]: df_new = pd.crosstab(df["Complaint Type"], df["City"])
df_new.head(5)
```

```
Out[59]:
```

	City	ARVERNE	ASTORIA	Astoria	BAYSIDE	BELLEROSE	BREEZY POINT	BRONX
Complaint Type								
Agency Issues		0	0	0	0	0	0	0
Animal Abuse		46	170	0	53	15	2	1971
Animal in a Park		0	0	0	0	0	0	0
Bike/Roller/Skate Chronic		0	16	0	0	1	0	22
Blocked Driveway		50	3436	159	514	138	3	17062

5 rows × 54 columns



4. Visualize the major types of complaints in each city

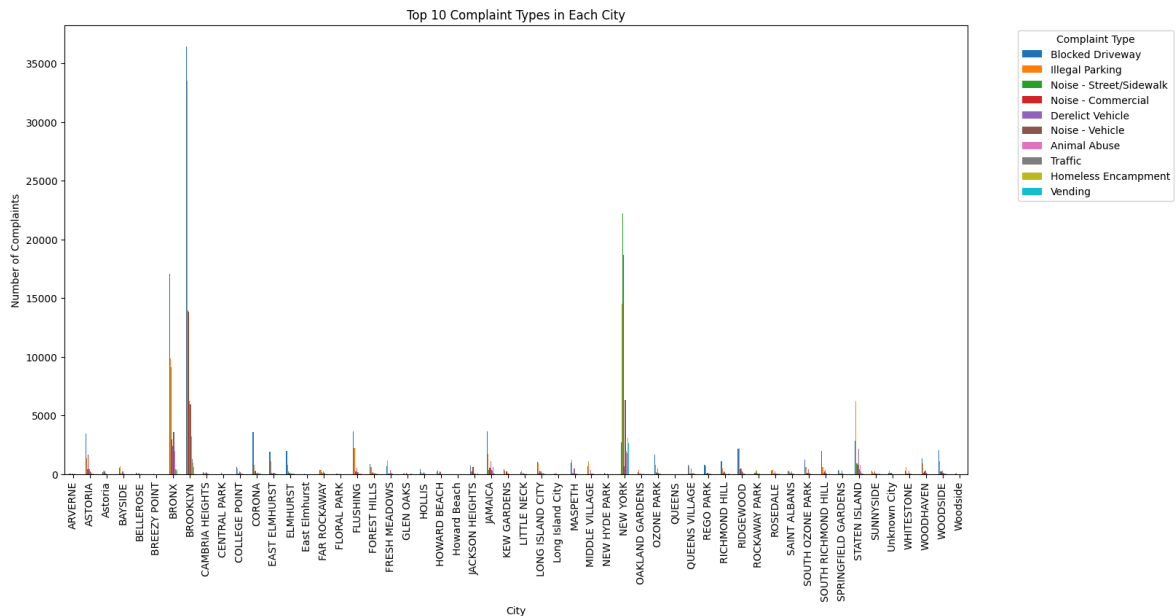
4.1 Draw another chart that shows the types of complaints in each city in a single chart, where different colors show the different types of complaints

```
In [88]: top10 = df["Complaint Type"].value_counts().head(10).index
city_top10 = pd.crosstab(df["City"], df["Complaint Type"])[top10]
city_top10.plot(
    kind='bar',
    figsize=(16,8)
)

plt.title("Top 10 Complaint Types in Each City")
plt.xlabel("City")
plt.ylabel("Number of Complaints")
```

```
plt.legend(
    title="Complaint Type",
    bbox_to_anchor=(1.05,1),
    loc='upper left'
)

plt.show()
```



4.2 Sort the complaint types based on the average Request_Closing_Timegrouping them for different locations

```
In [89]: avg_closing_time = df.groupby(["City", "Complaint Type"])["Request Closing Time"]
avg_closing_time.head()
```

```
Out[89]:
```

	City	Complaint Type	Request Closing Time Seconds
0	ARVERNE	Animal Abuse	8399.608696
1	ARVERNE	Blocked Driveway	8320.020000
2	ARVERNE	Derelict Vehicle	11391.687500
3	ARVERNE	Disorderly Youth	12928.500000
4	ARVERNE	Drinking	859.000000

```
In [92]: avg_closing_time = avg_closing_time.sort_values(
    by="Request Closing Time Seconds",
    ascending=False
)
avg_closing_time.head(10)
```

Out[92]:

	City	Complaint Type	Request Closing Time Seconds
510	QUEENS	Animal in a Park	1.212605e+06
525	QUEENS VILLAGE	Graffiti	1.920000e+05
63	BELLEROSE	Homeless Encampment	1.408800e+05
653	SOUTH RICHMOND HILL	Bike/Roller/Skate Chronic	1.112770e+05
482	OAKLAND GARDENS	Homeless Encampment	1.031400e+05
682	SPRINGFIELD GARDENS	Posting Advertisement	7.020650e+04
126	CAMBRIA HEIGHTS	Homeless Encampment	7.011833e+04
609	ROSEDALE	Homeless Encampment	6.790625e+04
438	MIDDLE VILLAGE	Bike/Roller/Skate Chronic	5.645600e+04
60	BELLEROSE	Derelict Vehicle	5.524882e+04

In [93]: `print(avg_closing_time.head(10))`

```

      City      Complaint Type \
510    QUEENS    Animal in a Park
525  QUEENS VILLAGE      Graffiti
63    BELLEROSE    Homeless Encampment
653  SOUTH RICHMOND HILL  Bike/Roller/Skate Chronic
482    OAKLAND GARDENS    Homeless Encampment
682  SPRINGFIELD GARDENS    Posting Advertisement
126    CAMBRIA HEIGHTS    Homeless Encampment
609      ROSEDALE    Homeless Encampment
438    MIDDLE VILLAGE  Bike/Roller/Skate Chronic
60    BELLEROSE    Derelict Vehicle

      Request Closing Time Seconds
510      1.212605e+06
525      1.920000e+05
63       1.408800e+05
653      1.112770e+05
482      1.031400e+05
682       7.020650e+04
126       7.011833e+04
609       6.790625e+04
438       5.645600e+04
60       5.524882e+04

```

5. See whether the average response time across different complaint types is similar (overall)

5.1 Visualize the average of Request_Closing_Time

In [101...]

```

avg_response_time = df.groupby("Complaint Type")["Request Closing Time Seconds"]
print(avg_response_time)

```

```

Complaint Type
Agency Issues          1.829512e+04
Animal Abuse            1.803257e+04
Animal in a Park        1.212605e+06
Bike/Roller/Skate Chronic 1.312373e+04
Blocked Driveway        1.623243e+04
Derelict Vehicle        2.535961e+04
Disorderly Youth        1.236330e+04
Drinking                1.382040e+04
Graffiti               2.327552e+04
Homeless Encampment     1.545133e+04
Illegal Fireworks       1.011255e+04
Illegal Parking         1.565054e+04
Noise - Commercial      1.108576e+04
Noise - House of Worship 1.139122e+04
Noise - Park            1.222613e+04
Noise - Street/Sidewalk 1.223129e+04
Noise - Vehicle         1.256187e+04
Panhandling             1.585314e+04
Posting Advertisement    7.285103e+03
Squeegee               1.456425e+04
Traffic                 1.230899e+04
Urinating in Public     1.295872e+04
Vending                 1.436650e+04
Name: Request Closing Time Seconds, dtype: float64

```

```

In [102... avg_response_time = avg_response_time.sort_values(ascending=False)
           print(avg_response_time)

```

```

Complaint Type
Animal in a Park        1.212605e+06
Derelict Vehicle        2.535961e+04
Graffiti               2.327552e+04
Agency Issues          1.829512e+04
Animal Abuse            1.803257e+04
Blocked Driveway        1.623243e+04
Panhandling             1.585314e+04
Illegal Parking         1.565054e+04
Homeless Encampment     1.545133e+04
Squeegee               1.456425e+04
Vending                 1.436650e+04
Drinking                1.382040e+04
Bike/Roller/Skate Chronic 1.312373e+04
Urinating in Public     1.295872e+04
Noise - Vehicle         1.256187e+04
Disorderly Youth        1.236330e+04
Traffic                 1.230899e+04
Noise - Street/Sidewalk 1.223129e+04
Noise - Park            1.222613e+04
Noise - House of Worship 1.139122e+04
Noise - Commercial      1.108576e+04
Illegal Fireworks       1.011255e+04
Posting Advertisement    7.285103e+03
Name: Request Closing Time Seconds, dtype: float64

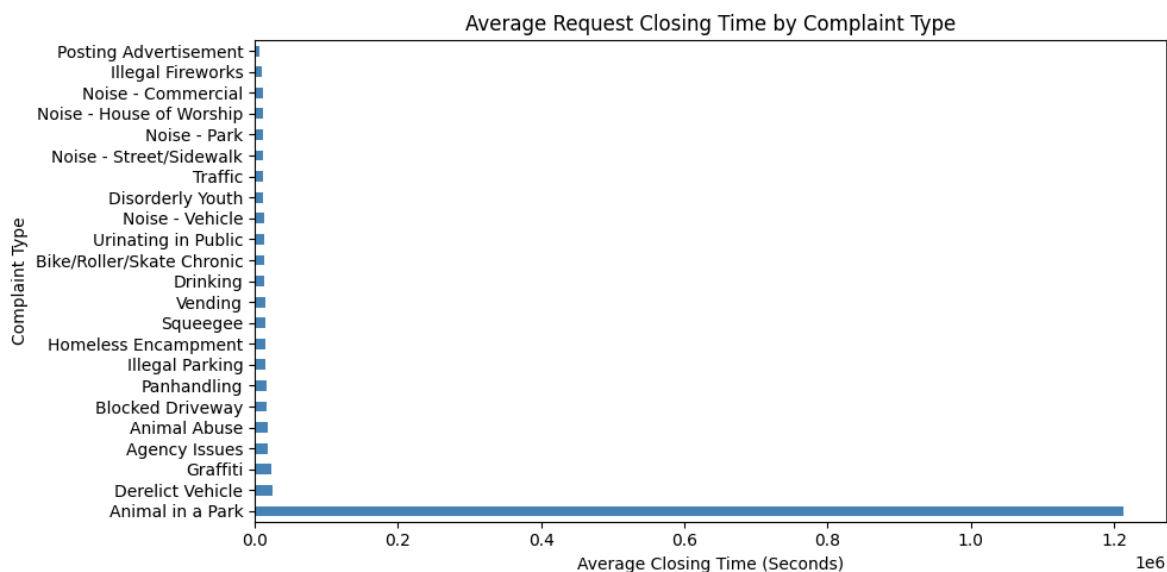
```

```

In [106... plt.figure(figsize=(10,5))
           avg_response_time.plot(
               kind="barh",
               color="steelblue"
           )

```

```
plt.title("Average Request Closing Time by Complaint Type")
plt.xlabel("Average Closing Time (Seconds)")
plt.ylabel("Complaint Type")
plt.tight_layout()
plt.show()
```



6. Identify the significant variables by performing statistical analysis using p-values

```
In [127... df["Created Date"] = pd.to_datetime(df["Created Date"], errors="coerce")
df["Closed Date"] = pd.to_datetime(df["Closed Date"], errors="coerce")
df["Request Closing Time"] = df["Closed Date"] - df["Created Date"]
df["Request Closing Time Seconds"] = df["Request Closing Time"].dt.total_seconds
print(df[["Request Closing Time", "Request Closing Time Seconds"]].head())
```

	Request Closing Time	Request Closing Time Seconds
0	0 days 00:55:15	3315.0
1	0 days 01:26:16	5176.0
2	0 days 04:51:31	17491.0
3	0 days 07:45:14	27914.0
4	0 days 03:27:02	12422.0

```
In [138... # Remove missing values
df_test = df.dropna(subset=["Complaint Type", "Request Closing Time Seconds"])
```

```
In [139... # Create Groups - Create one group for each complaint type.
groups = []
for complaint in df_test["Complaint Type"].unique():
    group = df_test[df_test["Complaint Type"] == complaint][["Request Closing Time Seconds"]]
    groups.append(group)
```

```
In [142... alpha = 0.05
if p_value < alpha:
    print("Reject the Null Hypothesis")
    print("Average response time differs significantly across complaint types.")
else:
    print("Fail to Reject the Null Hypothesis")
    print("Average response time is similar across complaint types.")
```

Reject the Null Hypothesis
Average response time differs significantly across complaint types.

7. Perform a Kruskal-WallisH test

7.1 Fail to reject H0: All sample distributions are equal

7.2 Reject H0: One or more sample distributions are not equal

```
In [143... # Perform the Kruskal-Wallis H Test
H_statistic, p_value = kruskal(*groups)
print("H Statistic:", H_statistic)
print("P-value:", p_value)
```

H Statistic: 5304.512660138937
P-value: 0.0

```
In [144... alpha = 0.05
if p_value >= alpha:
    print("Fail to Reject H0")
    print("All sample distributions are equal.")
else:
    print("Reject H0")
    print("One or more sample distributions are not equal.")
```

Reject H0
One or more sample distributions are not equal.